⊙ <u>Time Complexity</u> (T.C) :- It refers to the amount of time an algorithm takes to run as a function of input size (n).

1. Best Case
2. Average Case
3. $\boxed{\text{Worst Case}}$ ⇒ It is represented using Θ notation.

eg.- $\{\underline{20}, \underline{50}, \underline{30}, \underline{6}, \underline{4}\} \rightarrow 79, 4$

① In worst case, it had to explore all the elements and still the element was not found. So let's say if the size of the set is n, in that case all the n elements had to be checked whether it's equal to 79 or not.

- $\Theta(n)$ Input parameter

Offer $\Rightarrow \Theta(1)$

- Companies = {Meta, Apple, Amazon, Netflix, Google, Microsoft}

$\Theta(n)$

Time Taken: Best Case < Average Case < Worst Case

$$\Theta\left(\frac{n+1}{2}\right) = \Theta\left(\frac{n}{2} + \frac{1}{2}\right)$$

$$= \Theta\left(\frac{n}{2} + c\right)$$

where $c = \frac{1}{2}$

$$= c \cdot \Theta\left(\frac{n}{2}\right) = \frac{c}{2} \cdot \Theta(n)$$

(4) arr =

26	72	79	88	59	12
0	1	2	3	4	5

$i = i + 1$

$i = 5 + 1$

$= 6$

Let's say the size of the array is $n = 6$

```
for (int i = 0; i < n; i++)
{
    if (arr[i] == target)
        return i;
}// for loop closed
return -1;
```

6 < 6

target = 99

$\Theta(n)$

- In worst case, it is traversing the entire array hence T.C = $\Theta(n)$ where n is the size of the array.

- **Nested Loop :-** $n = 4$

```
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        // do some operation
    }// for j closed
}// for i closed
```

$j < 4$

- When $i = 0$; $j = 0, 1, 2, 3$ (4 times)

  When $i = 1$, $j = 1, 2, 3$ (3 times)

  When $i = 2$; $j = 2, 3$ (2 times)

  When $i = 3$ $j = 3$ (1 time)

$$n = 4 \quad (4 + 3 + 2 + 1) = 10$$

$$1 + 2 + 3 + 4 + 5 \cdots + n = \frac{n(n+1)}{2}$$

$$\Theta(n) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta\left(\frac{n^2+n}{2}\right)$$

ignored

$$= \Theta(n^2 + n)$$

10 000 0000

+ 10,000

$$\boxed{n = 10{,}000}$$

$$= \Theta(n^2)$$

- $\underline{1+2+3+\cdots+n} = \dfrac{n(n+1)}{2}$

- $\Theta(n) = \Theta(n^5 + \underline{\underline{n^3}} + 10)$

$$= \Theta(n^5)$$

10000

$\underline{n^{100}} > \underline{\underline{n^{99}}}$

func() ← $O(m+n)$

```
{ for (int i = 0; i < m; i++)
    {
      // do some operation
    }

  for (int j = 0; j < n; j++)
    { // do some operation
    }
} // function closed
```

$$\Theta(n^2 + n) = \Theta(n^2)$$

because $n^2 > n$ for Large value of n.

- $\boxed{\Theta(m + n) = \Theta(\max(m, n))}$

if $m > n = \Theta(m)$

if $(n > m) = \Theta(n)$

- $\Theta(c \cdot n) = \Theta(n)$

  where c is a constant

- $\Theta(n+n) = \Theta(2n)$

  $= \underline{\Theta(n)}$

- **Space Complexity** :- It refers to the amount of memory or storage space required by an algorithm to complete it's execution.

eg :-

```
void func ( int arr[n],
            int arr2 [m] )
{
    for (int i = 0; i < n; i++)
    {

    }
}
```

$\Rightarrow SC = \Theta(n+m)$

$= \Theta(\max(n, m))$

```
void func2( int arr1(n), int arr2(m))
{
    int vis(n)(m);
```

$\Theta(n)$ $\Theta(m)$

$\Theta(n*m)$ → $n = 2$, $m = 3$

6, 2, 3

} // closed

$$\boxed{S.C = \Theta(n*m)}$$

$T.C = \Theta(1)$

	0	1	2	3	4	
0	3	4	5	6	7	5
1	8	9	10	11	12	5
2	13	14	15	16	17	5

$n = 3, \quad m = 5$

3×5

$= 15$